Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

# Дискретна математика

# Лабораторна робота №5

«Комбінаторика: перестановки, розміщення, сполучення».

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 18
Перевірив:
Пономаренко А. М.

Київ – 2025

# <u>Лабораторна робота №5</u>

**<u>Тема:</u>** «Комбінаторика: перестановки, розміщення, сполучення».

**<u>Мета:</u>** вивчення правил утворення комбінацій множин: *перестановок, розміщень, сполучень*.

**<u>Загальне завдання:</u>**

1. Вивчити алгоритми формування перестановок, сполучень та розбиття. Написати програми для виконання даних алгоритмів.

**<u>Теоретичні основи:</u>**

*Комбінації з n елементів, які відрізняються одна від одної тільки порядком елементів, називають перестановками.*

Перестановки позначають символом $P_n$, де $n$ — число елементів, що входять у кожну перестановку.

**Приклад.** Нехай множина $M$ містить три букви $A$, $B$, $C$. Складемо всі можливі комбінації із цих букв: $ABC$, $ACB$, $BCA$, $CAB$, $CBA$, $BAC$ (усього 6 комбінацій).

Видно, що вони відрізняються одна від одної тільки порядком розташування букв.

*Добуток всіх натуральних чисел від 1 до n включно називають n-факторіалом* і пишуть: $n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot (n-1) \cdot n$. Вважають, що $0! = 1$ і $n \in N$. Основна властивість факторіала: $(n+1)! = (n+1) \cdot n!$.

Отже, число перестановок обчислюємо за формулою: $P_n = n!$.

*Комбінації з n елементів по m елементів, які відрізняються одна від одної або самими елементами, або порядком елементів, називають розміщеннями.*

Розміщення позначають символом $A_n^m$, де $n$ – число всіх наявних елементів, $m$ – число елементів у кожній комбінації. Число розміщень можна обчислити за формулою:

$$A_n^m = n(n-1)(n-2)\ldots(n-m+1), \text{ де } 0 \le m \le n;\ m, n \in N.$$

Вважають, що $A_n^0 = 1$.

**Приклад.** Нехай множина $M$ містить чотири букви $A$, $B$, $C$, $D$. Склавши всі комбінації тільки із двох букв, одержимо: $AB$, $AC$, $AD$, $BA$, $BP$, $BD$, $CA$, $CB$, $CD$, $DA$, $DB$, $DC$.

Формулу розміщення можна записати у факторіальній формі:

$$A_n^m = \frac{n!}{(n-m)!}.$$

Основні властивості розміщень:

1) $A_n^{m+1} = A_n^m \cdot (n-m)$;

2) $A_n^n = P_n = n!$ .

*Сполученнями називають всі можливі комбінації з n елементів по m, які відрізняються одна від одної принаймні хоча б одним елементом ( m, n ∈ N і n ≥ m).*

У загальному випадку число сполучень із n елементів по m дорівнює числу розміщень з n елементів по m, діленому на число перестановок з m елементів:

$$C_n^m = \frac{A_n^m}{P_m}$$

Використовуючи для кількості розміщень і перестановок факторіальні формули $A_n^m = \frac{n!}{(n-m)!}$ і $P_n = n!$, одержимо формулу кількості сполучень у вигляді:

$$C_n^m = \frac{n!}{(n-m)!\ m!}$$

Основні властивості сполучень: $C_n^{n-m} = \frac{P_n}{P_{n-m} \cdot P_m} = \frac{n!}{(n-m)!\ m!}$; $C_n^m = C_n^{n-m}$

**Приклад.** Множина M утворена з чотирьох букв A, B, C, D. Скласти комбінації з двох букв, що відрізняються хоча б одним елементом. Маємо AB, AC, AD, BP, BD, CD. Виходить, що кількість сполучень з чотирьох елементів по два дорівнює 6. Це коротко записують так: $C_4^2 = 6$.

Розміщення з *n* елементів по *k* відображають упорядковані комбінації різних елементів множини *M*, |*M*|=*n*. Часто доводиться утворювати упорядковані комбінації з повтореннями деяких елементів. Наприклад, з множини *M* = {*А, Б*} можна утворити вісім комбінацій з трьох елементів: ААА, ААБ, АБА, БАА, БАБ, ББА, АББ, БББ. Тут n = 2, k = 3. Такі упорядковані k-комбінації називають кортежами довжини k. Два кортежі (тобто дві загальні комбінації) вважають однаковими, якщо вони мають однакову довжину і на місцях з однаковими номерами стоять однакові елементи.

*Кортеж довжини k з n елементів називають розміщенням з повтореннями з n елементів по k .*

Кількість кортежів обчислюють за формулою: $\overline{A_n^k} = n^k$.

Дійсно, після заповнення першого місця кортежу довжиною k одним з n елементів (що можливо зробити n варіантами) заповнити друге місце кортежу можна знову будь-яким елементом з усієї множини (повторюючи в одному з варіантів елемент, який знаходиться на першому місці), і так далі k разів. За правилом добутку одержимо, що $\overline{A_n^k} = \underbrace{n \cdot n \cdot \ldots \cdot n}_{k \text{ разів}} = n^k$.

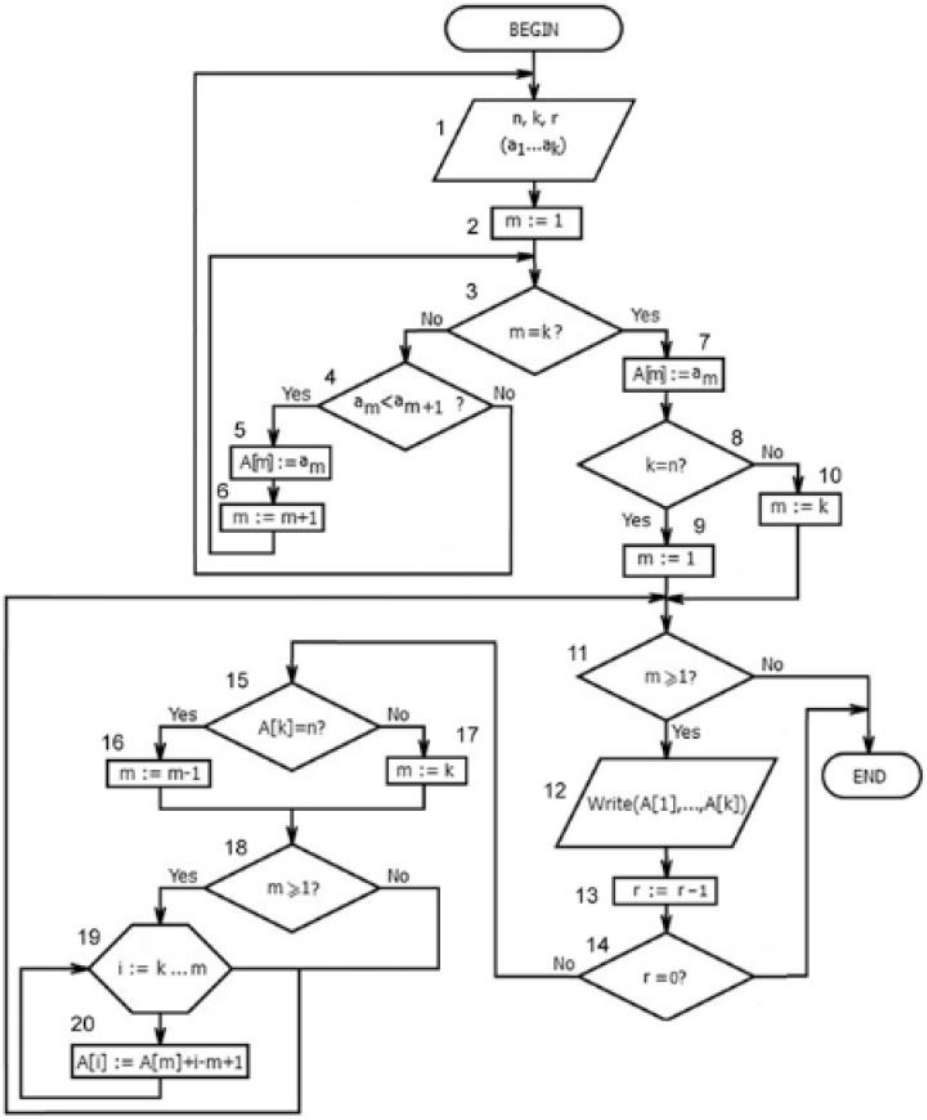


Рисунок 1.1 – генерація сполучень з n по k в лексикографічному порядку

**Блок 1.** Ввід початкових даних: $n$ - довжина початкової послідовності, $k$ - кількість елементів сполучення, $r$ - кількість сполучень, які необхідно згенерувати, $(a_1, a_2, ..., a_k)$ - початкове сполучення, з якого алгоритм починає генерувати наступні сполучення у лексикографічному порядку, $a_1 < a_2 < ... < a_k$ - умова задавання початкового сполучення.

**Блок 2.** Установка в початковий стан лічильніка кількості елементів початкового сполучення $m$:= 1.

**Блок 3.** Перевірка ознаки закінчення оцінки умови правильності задавання елементів початкового сполучення.

**Блок 4.** Блок виконується у випадку, коли ще не всі елементи початкового сполучення перевірені. В цьому випадку порівнюється поточний елемент початкового сполучення з наступним елементом. Якщо $a_m < a_{m+1}$ , то елемент $a_m$ задано правильно.

**Блок 5.** Значення елемента початкового сполучення $am$ записуються в $m$-й елемент масиву $A[m]:= a_m$ .

**Блок 6.** Інкремент лічильника кількості елементів початкового сполучення $m:= m+1$.

**Блок 7.** Якщо лічильник $m$ досяг максимально допустимого значення, тобто $m = k$ , записуємо цей останній елемент в масив: $A[m]:= a_m$ .

*Увага! Далі змінна m буде використовуватися в новому смисловому значенні. Тепер вона виконує роль ознаки закінчення генерації всіх можливих сполучень з n по k .*

**Блок 8.** Перевіряємо випадок, коли довжина сполучення дорівнює довжині початкової послідовності, тобто чи $n = k$ .

**Блок 9.** Якщо $n = k$ , то змінній $m$ надаємо значення 1.

**Блок 10.** Якщо $k < n$ , то $m:= k$ .

**Блок 11.** Перевіряємо ознаку закінчення роботи алгоритму $m$. Якщо $m \geq 1$, то переходимо до роздруківки чергового сполучення. В протилежному випадку закінчуємо роботу алгоритму.

**Блок 12.** Роздруківка масиву поточного сполучення $A$.

**Блок 13.** Декремент лічильника кількості замовлених сполучень. Цей блок використовують у випадку, коли потрібно згенерувати наперед задану обмежену кількість сполучень. Якщо алгоритм повинен згенерувати всі можливі сполучення, починаючи з початкового, то задають $r > C_n^k$.

**Блок 14.** Перевірка ознаки кількості замовлених сполучень. Якщо $r = 0$, то всі замовлені сполучення згенеровано і алгоритм закінчує роботу. В протилежному випадку відбувається перехід до наступної частини алгоритму, яка забезпечує формування нового сполучення.

**Блоки 15-20.** Алгоритм генерації чергового сполучення.

**Загальний порядок виконання роботи:**

A. Вибрати номер Z індивідуального варіанта відповідно до виразу:

*I = NZK mod26+1, де NZK – номер залікової книжки.*

B. Програма обчислення варіанта:

```
G=4218
print("Номер залікової книжки:",G)
print("Мій варіант:",(G%26) +1)
```

**Індивідуальне завдання:**

| № вар | Опис варіанту |
|---|---|
| 7 | Вивчити принципи роботи алгоритму генерації двійкових векторів довжини *n*. Написати програму генерації двійкових векторів довжини *n* у лексикографічному порядку.<br>Вхідні параметри мають такі значення:<br>1. Максимальне значення *n* дорівнює номеру залікової книжки (NZK).<br>2. Значення *m* може змінюватися довільно від 1 до *n* .<br>3. Сформувати початкову перестановку (*b*[*n*], ...,*b*[0]) таким чином, щоб кожен її елемент вибирався випадково. |

**Розрахунок варіанту:**

**Код програми:**

```
G=4218
print("Номер залікової книжки: ",G)
print("Мій варіант:",(G%26) +1)
```

**Результат виконання програми:**

```
Номер залікової книжки: 4218
Мій варіант: 7
```

Рисунок 2.1 – результат обрахунку варіанта

**Код програми:**

**Вміст файлу LR5.py:**

```
import tkinter as tk
from tkinter import messagebox
import random
import varCheck as variant

class BinaryVectorApp:
    def __init__(self, master):
        self.master = master
        master.title("ЛР5")
```

```
        frame = tk.Frame(master)
        frame.pack(padx=10, pady=10)
        tk.Label(frame, text=(f"Куліков Максим Миколайович\nГрупа: ІО-{variant.G}\nНомер у
списку: {variant.N}\nВаріант: {variant.get_variant_number()}")).pack(pady=(0, 10))
        tk.Label(frame, text="Введіть n (довжина вектора):").pack(anchor="w")
        self.entry_n = tk.Entry(frame)
        self.entry_n.pack(fill="x")
        tk.Label(frame, text="Введіть m (кількість векторів):").pack(anchor="w")
        self.entry_m = tk.Entry(frame)
        self.entry_m.pack(fill="x", pady=(0, 10))
        tk.Button(frame, text="Згенерувати вектори", command=self.run_algorithm).pack(fill="x",
pady=5)
        tk.Label(frame, text="Результат:").pack(anchor="w")
        self.output_text = tk.Text(frame, height=15, width=50)
        self.output_text.pack()
    def run_algorithm(self):
        try:
            n = int(self.entry_n.get())
            m = int(self.entry_m.get())
            if n <= 0 or m <= 0 or n > variant.get_NZK() or m > n:
                raise ValueError
        except:
            messagebox.showerror("Помилка", f"Введіть коректні числа n і m (n ≤
{variant.get_NZK()}).")
            return
        b = [random.randint(0, 1) for _ in range(n)]
        result = ["".join(map(str, b))]
        for _ in range(m - 1):
            i = n - 1
            while i >= 0 and b[i] == 1:
                b[i] = 0
                i -= 1
            if i < 0:
                break
            b[i] = 1
            result.append("".join(map(str, b)))
        self.output_text.delete("1.0", "end")
        self.output_text.insert("1.0", "\n".join(result))

if __name__ == "__main__":
    root = tk.Tk()
    app = BinaryVectorApp(root)
    root.mainloop()
```

## Вміст файлу varCheck.py:

```
G = 42
N= 18

def get_variant_number():
    return (int(str(G) + str(N))%26)+1
def print_variant_number():
    print("Номер залікової книжки:",int(str(G)+str(N)))
    print("Мій варіант:",get_variant_number())
def get_NZK():
    return (int(str(G) + str(N)))

if __name__ == "__main__":
    print_variant_number()
```

**Результат виконання програми:**

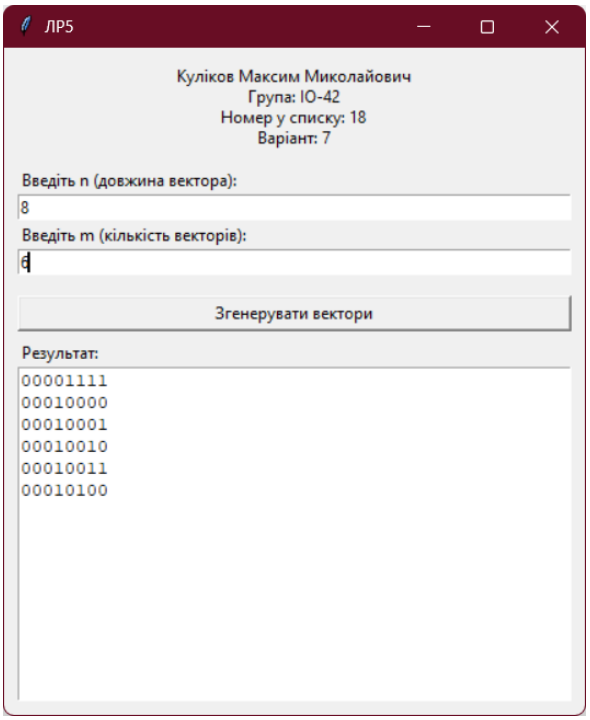


Рисунок 3.1 – результат виконання програми

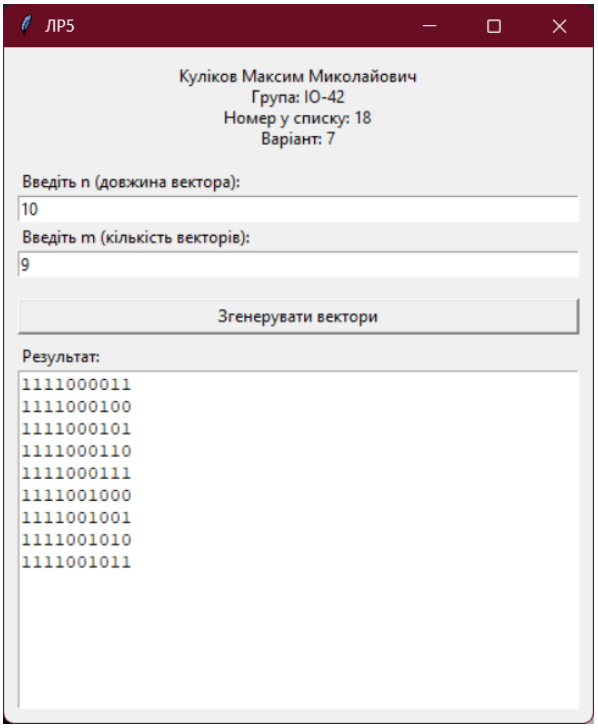


Рисунок 3.2 – результат виконання програми з іншими числами

**Висновки:**

Отже, під час виконання лабораторної роботи я створив програму генерації двійкових векторів довжини n у лексикографічному порядку. Під час виконання проблем не виникло.